

AssureX Claim Engine

Python Classification Model Report

Group	NN_DevStorm
Batch	2609E2'
Faculty	Sir Minhaj
Repository	https://github.com/Saba1512006/assurex-claim-engine
Live application	https://assurex.pythonanywhere.com
Date	29 September 2026

Student ID	Name	Role
Student1525913	Saba Noor	Team lead · integration, frontend direction, QA, documentation
Student1524867	Muhammad Sami	Backend and ML pipeline developer · deployment
Student1509222	Muhammad Ghanyan	Frontend design (Figma, colour palette, mobile checks)
Student1505677	Muhammad Sami ur Rehman	Testing (sample claims, Chrome and mobile)

Abstract

AssureX Claim Engine evaluates warranty claims for consumer electronics, home appliances and industrial tools. A customer or service center registers a product from its receipt (read by OCR), files a claim with evidence, and the application pre-processes the claim into model features. A calibrated Python classification model (HistGradientBoosting, 89.3% test accuracy on 225 unseen claims) scores the structured record; the same claim is drawn as a Claim Summary Card and classified by a Google Teachable Machine model (93.8% test accuracy). The two predictions and their confidences are compared (Strong Match, Acceptable Match, Weak Match, Model Disagreement, Uncertain Result), the category's warranty policy is executed from JSON configuration, contradictions, missing documents and duplicates are detected, and a decision table produces Likely Valid, Likely Invalid or Manual Review Required. Anything uncertain goes to a human reviewer, and every step is recorded in an audit trail. The application has four roles (customer, service-center staff, claim reviewer, administrator), 295 automated tests and 34 browser tests.

Complete evidence for the Python classification model (SRS §1.10 item 4): data, pre-processing, the three algorithms compared, hyper-parameters, cross-validation, validation and test results, confusion matrix, per-class metrics, feature importance, saved files and sample predictions.

1. Python classification model evidence

Source of every number below: [model/python_model/model_card_v2.json](#), written by [notebooks/train_python_v2.py](#) (seed 42, deterministic — re-running gives the same artefact hash 514e2fb68286...). The live values are also shown in the app under Admin > Analytics.

Data

Item	Value
Dataset	data/raw/common_warranty_claims_1500.csv — 1,500 claims, 500 per class
Split (stratified, before any card was rendered)	train 1,050 · validation 225 · test 225, 350/75/75 per class
Isolation	a claim ID belongs to exactly one split (asserted in the generator and by tests/test_ml_integrity.py)
Label noise	4% of labels flipped on purpose (reviewer disagreement) → realistic ceiling ≈ 96%

Features (20) and pre-processing

Group	Features	Handling
Numeric (8)	purchase_price, warranty_duration_months, product_age_days, days_to_expiry, reporting_delay_days, diagnostic_confidence, previous_repairs_count, missing_document_count	Standardised (StandardScaler) for the linear model; raw for tree models
Binary (10)	is_extended_warranty, has_receipt, has_warranty_card, has_damage_photo, has_serial_photo, has_repair_report, serial_number_match, unauthorized_repair_flag, duplicate_invoice_flag, claim_date_conflict_flag	Passed through as 0/1
Categorical (2)	product_category (3 values), damage_type (7 values)	One-hot with the fixed vocabulary from src/core/vocab.py ; an unseen value raises an error instead of being silently zeroed

Never features: claim_id, scenario, claim_class, fault_description, identifiers (asserted in the trainer).

Missing values. In the web app, absent documents become 0, no repairs 0, and an unassessed diagnosis takes the neutral prior 0.5 (the mean of the non-defect training distribution). Categories outside the vocabulary are rejected with a message. **Derived fields:** product age, days to expiry, remaining warranty, reporting delay, missing-document count, conflict / mismatch / duplicate flags — [src/core/features.py](#) builds them with exactly the training column names.

Encoder / pre-processing files. The application loads one artefact, [model/python_model/claim_classifier_v2.joblib](#), which contains the pre-processing and the model together so they can never drift apart. [python notebooks/export_model_artifacts.py](#) extracts the pieces from that saved model (nothing is refitted) and a test checks they match it exactly:

File	Content
model/python_model/preprocessing_pipeline.joblib	ColumnTransformer : numeric and binary passthrough, one-hot for category and damage type with the fixed vocabulary
model/python_model/label_encoder.joblib	LabelEncoder for Invalid Claim / Manual Review / Valid Claim (ids 0 / 1 / 2)

File	Content
model/python_model/feature_schema.json	Input columns and types, vocabularies, missing-value and encoding rules, the 30 encoded output columns
data/processed/{train,val,test}_features.csv	Model-ready matrices: claim ID, 30 encoded features, class and label id
data/labels.csv, data/claim_scenarios.json	Label, split, scenario and card files per claim; scenario definitions and counts

Leakage audit

A depth-3 decision tree trained on **one feature at a time** (validation accuracy; chance = 33%):

Feature	Accuracy alone
days_to_expiry	52.9%
product_age_days	49.8%
missing_document_count	45.3%
has_receipt	42.7%
unauthorized_repair_flag	40.9%

The trainer refuses to continue if any single feature exceeds 90% (v1's `damage_type` was a renamed label and scored 100%).

Algorithms tested (5-fold stratified CV on train, then the validation split)

Algorithm	Hyper-parameters	CV macro-F1	Val accuracy	Val precision	Val recall	Val macro-F1
Logistic regression	C=1.0, max_iter=2000, scaled inputs	0.739 ± 0.026	72.9%	73.3%	72.9%	72.6%
Random forest	400 trees, min_samples_leaf=2, class_weight=balanced	0.922 ± 0.018	94.2%	94.2%	94.2%	94.2%
HistGradientBoosting (selected)	max_iter=300, learning_rate=0.06, max_leaf_nodes=15, l2=1.0	0.913 ± 0.008	94.7%	94.7%	94.7%	94.7%

Selection uses the **validation** macro-F1 only; the test split is touched once, at the end. The selected pipeline is then wrapped in `CalibratedClassifierCV(method="sigmoid", cv=5)` so its probabilities are usable as confidence scores (the consistency statuses depend on them).

Test results (225 unseen claims)

Metric	Value
Accuracy	89.3% (SRS NFR-4 target 85%)
Macro precision / recall / F1	89.5% / 89.3% / 89.2%
ROC-AUC (one-vs-rest, macro)	0.948
Log-loss	0.406
Expected calibration error	0.082
Mean top-class confidence	82.6%

Confusion matrix (rows actual, columns predicted):

	Invalid	Manual Review	Valid
Invalid Claim	61	8	6
Manual Review	2	69	4
Valid Claim	3	1	71

Class	Precision	Recall	F1	Support
Invalid Claim	92.4%	81.3%	86.5%	75
Manual Review	88.5%	92.0%	90.2%	75
Valid Claim	87.7%	94.7%	91.0%	75

The weakest cell is Invalid → Manual Review (8). Checking the 14 missed Invalid claims by scenario: 5 are *late reporting* (a rare scenario — 23 of 1,500 claims — so the model has seen few examples), 4 carry deliberately flipped labels (noise), 3 are *excluded damage* with a diagnosis near the 0.55 threshold, and 2 are expiry/grace boundary cases. All of these are claims the rule engine decides deterministically (LATE_REPORTING, EXCLUDED_DAMAGE, WARRANTY_EXPIRED are hard-fail rules), which is why the application never relies on the model alone.

Feature importance (permutation, validation split, drop in macro-F1)

days_to_expiry 0.279 · damage_type 0.161 · unauthorized_repair_flag 0.098 · diagnostic_confidence 0.081 · has_receipt 0.076 · serial_number_match 0.057 · missing_document_count 0.056 · reporting_delay_days 0.054

These match the warranty policy: expiry, excluded causes, unauthorised repairs and proof of purchase drive the outcome.

Model version and sample predictions

- Version string stored with every prediction: `v2.0.0+<first 12 hex of the file's SHA-256>`.
- scikit-learn 1.9.1 (pinned in `requirements.txt` so the pickle loads identically everywhere).
- 30 sample test predictions with all three probabilities: `model_card_v2.json` → `test.sample_predictions`; all 225 in `model/python_model/sample_test_predictions.csv` and `reports/model_comparison_report.csv`.

Feature importance (permutation, validation split)

Feature	Importance (drop in macro-F1)
days_to_expiry	0.2788
damage_type	0.1606
unauthorized_repair_flag	0.098
diagnostic_confidence	0.0805
has_receipt	0.076
serial_number_match	0.0565
missing_document_count	0.0562
reporting_delay_days	0.0543
duplicate_invoice_flag	0.0304
previous_repairs_count	0.0219

Feature	Importance (drop in macro-F1)
product_category	0.0124
warranty_duration_months	0.0106
purchase_price	0.0102
product_age_days	0.0097
is_extended_warranty	0.0022
has_damage_photo	0.0013
has_serial_photo	0.0009
claim_date_conflict_flag	0.0009
has_warranty_card	0.0
has_repair_report	0.0

Sample test predictions (first 25 of the 225 test claims)

Full list: model/python_model/sample_test_predictions.csv.

claim_id	actual_class	predicted_class	p_valid	p_invalid	p_manual_review	correct
CLM-01428	Manual Review	Manual Review	0.02	0.0393	0.9407	True
CLM-00367	Valid Claim	Valid Claim	0.7903	0.0957	0.114	True
CLM-00654	Valid Claim	Valid Claim	0.73	0.0355	0.2345	True
CLM-00539	Invalid Claim	Invalid Claim	0.0119	0.8961	0.092	True
CLM-00816	Manual Review	Manual Review	0.0271	0.2208	0.7521	True
CLM-01383	Valid Claim	Valid Claim	0.8138	0.0412	0.1449	True
CLM-00508	Valid Claim	Valid Claim	0.6899	0.0157	0.2943	True
CLM-00850	Valid Claim	Valid Claim	0.8673	0.0182	0.1145	True
CLM-00535	Invalid Claim	Invalid Claim	0.0478	0.8358	0.1165	True
CLM-00988	Invalid Claim	Valid Claim	0.9376	0.0305	0.0319	False
CLM-00458	Manual Review	Manual Review	0.0167	0.0853	0.8981	True
CLM-00266	Manual Review	Manual Review	0.1796	0.071	0.7494	True
CLM-00159	Valid Claim	Valid Claim	0.8131	0.0949	0.0919	True
CLM-00925	Invalid Claim	Invalid Claim	0.0123	0.8131	0.1746	True
CLM-01117	Valid Claim	Valid Claim	0.7763	0.0452	0.1785	True
CLM-01444	Invalid Claim	Invalid Claim	0.0244	0.8322	0.1434	True
CLM-01196	Valid Claim	Valid Claim	0.7772	0.0127	0.2102	True
CLM-01441	Invalid Claim	Invalid Claim	0.0086	0.7925	0.1989	True
CLM-00906	Valid Claim	Valid Claim	0.8365	0.0215	0.142	True
CLM-00627	Invalid Claim	Invalid Claim	0.0055	0.9561	0.0385	True
CLM-00016	Manual Review	Manual Review	0.0064	0.3826	0.611	True
CLM-01154	Valid Claim	Valid Claim	0.9429	0.0238	0.0333	True
CLM-00104	Manual Review	Manual Review	0.1524	0.0086	0.8391	True
CLM-00066	Valid Claim	Valid Claim	0.7844	0.0144	0.2011	True
CLM-00078	Invalid Claim	Invalid Claim	0.0369	0.9013	0.0618	True

Saved model files (model/python_model/)

File	Size
claim_classifier_v2.joblib	8069 KB

File	Size
feature_schema.json	2 KB
label_encoder.joblib	0 KB
model_card_v2.json	11 KB
preprocessing_pipeline.joblib	4 KB
sample_test_predictions.csv	14 KB